

Programming for Analytics

- SQL 2

Pedro Abreu (pabreu@pbs.up.pt)

ACCREDITATIONS



MEMBERSHIPS



RANKINGS



Overview of the class

Sub-Queries

CREATE, INSERT, UPDATE AND DELETE

Window functions

Text2SQL

Quizz



Evaluation

Evaluation

Component	Weight	Details
-----------	--------	---------

Quizzes	20%	One per session (sessions 1–4), administered at the end of class (last 10 minutes) via In Virtual Campus. 5 points each.
---------	-----	--

Group Project	40%	Continuous project developed across all sessions. Final deliverables submitted after Session 5: code and slide presentation. Mandatory.
---------------	-----	--

Exam	40%	Covers all course content: 50% SQL / 50% Python. Held in the last hour of Session 5. Mandatory.
------	-----	--

Important Rules

- Both the **project** and the **exam** are mandatory components. Failure to submit or sit either will result in course failure.
- Students who pass in the normal evaluation period are **not eligible** for the extraordinary evaluation period.
- **Extraordinary evaluation:** exam only (40% of the grade). Neither the project (40%) nor the quizzes (20%) can be recovered through extraordinary evaluation.

Quizzes and mini-test are individual and you cannot use generative models (like Chatgpt, Perplexity, Copilot or others)

Previous class...

Example (go to MySQL Workbench)

SELECT

```
o.orderNumber,  
o.status,  
od.quantityOrdered,  
od.priceEach,  
od.orderNumber
```

```
FROM classicmodels.Orders o
```

```
INNER JOIN classicmodels.OrderDetails od ON o.orderNumber=od.orderNumber;
```

SELECT

column1, column2, ...

Columns to retrieve

FROM

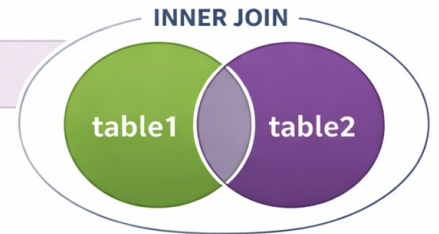
table1

First table

JOIN

table2

This query



ON

table1.column = table2.column

Join Condition

SQL – SubQueries

a subquery can be defined as a query embedded within another query.

```
SELECT
o.orderNumber,
o.status,
od.quantityOrdered,
od.priceEach,
od.orderNumber
FROM classicmodels.Orders o
INNER JOIN classicmodels.OrderDetails od ON o.orderNumber=od.orderNumber;
```

What if just want this information for orders already shipped?

SQL – SubQueries

a subquery can be defined as a query embedded within another query.

```
SELECT
o.orderNumber,
o.status,
od.quantityOrdered,
od.priceEach,
od.orderNumber
FROM classicmodels.Orders o
INNER JOIN classicmodels.OrderDetails od ON o.orderNumber=od.orderNumber;
```

What if just want this information for orders already shipped?

a subquery can be defined as a query embedded within another query.

SELECT

o.orderNumber,

o.**status**,

od.quantityOrdered,

od.priceEach,

od.orderNumber

FROM classicmodels.Orders o

INNER JOIN classicmodels.OrderDetails od **ON** o.orderNumber=od.orderNumber;

What if just want this information for orders already shipped?

SQL – SubQueries

a subquery can be defined as a query embedded within another query.

SELECT

o.orderNumber,

o.status,

od.quantityOrdered,

od.priceEach,

od.orderNumber

FROM classicmodels.Orders o

INNER JOIN classicmodels.OrderDetails od **ON** o.orderNumber=od.orderNumber;

What if just want this information for orders already shipped?


```
SELECT *  
FROM classicmodels.Orders o  
INNER JOIN classicmodels.OrderDetails od ON od.orderNumber=so.orderNumber;
```

```
SELECT *  
FROM (SELECT * FROM classicmodels.Orders so WHERE status = "Shipped") so  
inner join classicmodels.OrderDetails od ON od.orderNumber=so.orderNumber;
```

SQL – SubQueries (using With)

Using WITH	Not using WITH
WITH cte1 AS (SELECT a, b FROM table1), cte2 AS (SELECT c, d FROM table2) SELECT b, d FROM cte1 JOIN cte2 WHERE cte1.a = cte2.c;	SELECT b, d FROM (SELECT a, b FROM table1) cte1 JOIN (SELECT c, d FROM table2) cte2 WHERE cte1.a = cte2.c;

SELECT *

FROM (SELECT * FROM classicmodels.Orders so WHERE status = "Shipped") so
inner join classicmodels.OrderDetails od **ON** od.orderNumber=so.orderNumber;

WITH

shipped_orders **AS** (SELECT * FROM classicmodels.Orders o WHERE status = "Shipped")

SELECT *

FROM shipped_orders so

INNER JOIN classicmodels.OrderDetails od **ON** od.orderNumber=so.orderNumber;

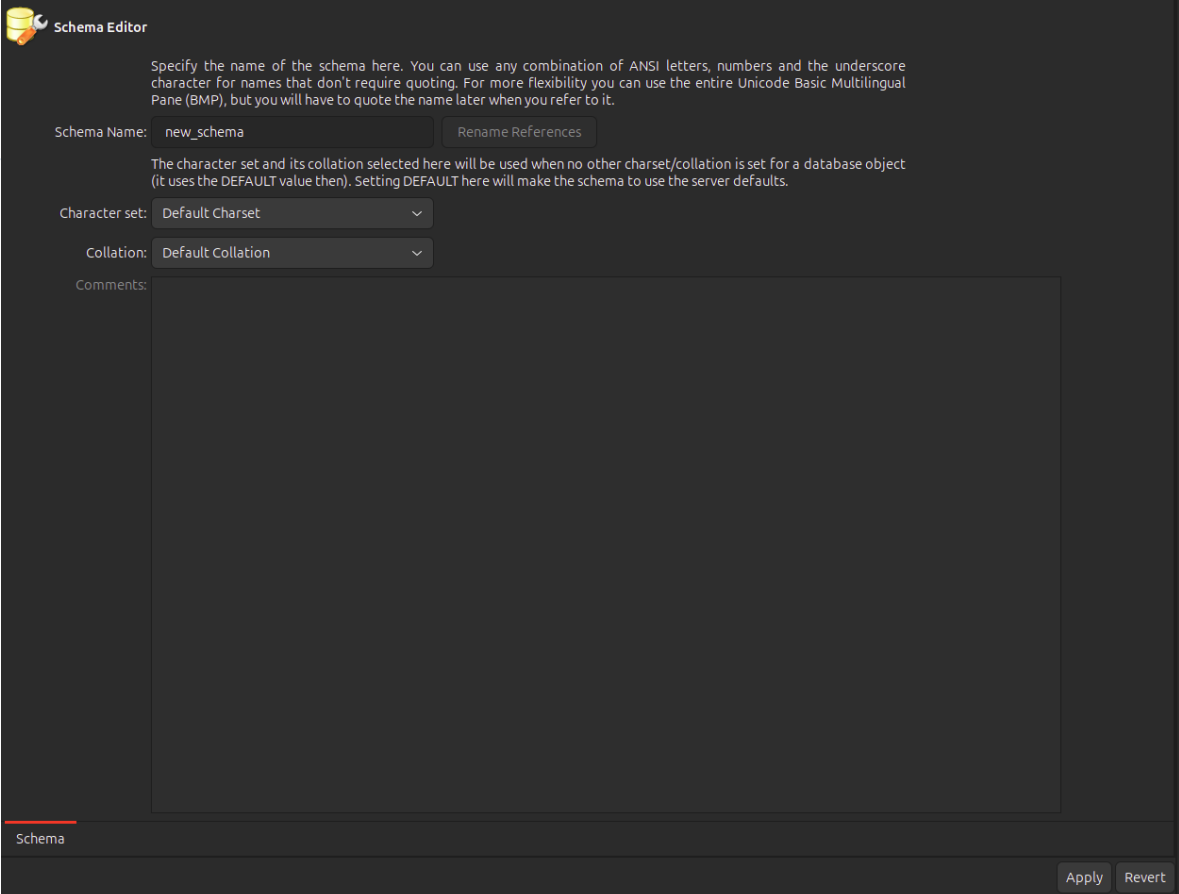
Exercises

- 1) Join table Products and ProductsLines, only with productLine with value “Ships”, with a left join
- 2) Join table Payments and Customers, from USA country, with a left join using customerNumber column

CREATE SCHEMA

CREATE SCHEMA myschema;

Create a schema, where we can create tables, manage permissions, etc



The screenshot shows a 'Schema Editor' window with a dark theme. At the top, there's a title bar with a yellow icon and the text 'Schema Editor'. Below the title bar, there's a text area with instructions: 'Specify the name of the schema here. You can use any combination of ANSI letters, numbers and the underscore character for names that don't require quoting. For more flexibility you can use the entire Unicode Basic Multilingual Pane (BMP), but you will have to quote the name later when you refer to it.' Below this, there's a 'Schema Name' field with the value 'new_schema' and a 'Rename References' button. Underneath, there's a text area with instructions: 'The character set and its collation selected here will be used when no other charset/collation is set for a database object (it uses the DEFAULT value then). Setting DEFAULT here will make the schema to use the server defaults.' Below this, there's a 'Character set' dropdown menu with 'Default Charset' selected, and a 'Collation' dropdown menu with 'Default Collation' selected. At the bottom, there's a 'Comments' text area. The bottom of the window has a 'Schema' tab and 'Apply' and 'Revert' buttons.

Schema Editor

Specify the name of the schema here. You can use any combination of ANSI letters, numbers and the underscore character for names that don't require quoting. For more flexibility you can use the entire Unicode Basic Multilingual Pane (BMP), but you will have to quote the name later when you refer to it.

Schema Name: new_schema [Rename References](#)

The character set and its collation selected here will be used when no other charset/collation is set for a database object (it uses the DEFAULT value then). Setting DEFAULT here will make the schema to use the server defaults.

Character set: Default Charset

Collation: Default Collation

Comments:

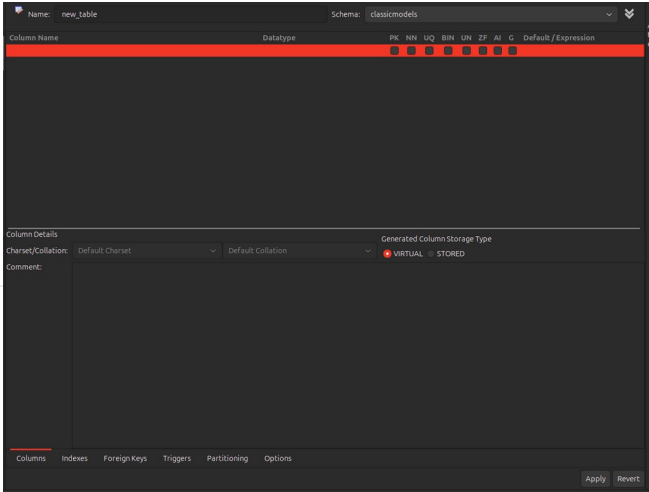
Schema

[Apply](#) [Revert](#)

CREATE TABLE

Create a new table

```
CREATE TABLE `Customers` (  
  `customerNumber` int NOT NULL,  
  `customerName` varchar(50) NOT NULL,  
  `contactLastName` varchar(50) NOT NULL,  
  `contactFirstName` varchar(50) NOT NULL,  
  `phone` varchar(50) NOT NULL,  
  `addressLine1` varchar(50) NOT NULL  
)
```



Types	Description
TEXT	Holds a string with a maximum length of 65,535 bytes
VARCHAR(size)	A VARIABLE length string (can contain letters, numbers, and special characters). The size parameter specifies the maximum column length in characters - can be from 0 to 65535
FLOAT	A floating point number. The total number of digits is specified in size. The number of digits after the decimal point is specified in the d parameter.
INT	A normal-size integer. The signed range is -2147483648 to 2147483647. The unsigned range is 0 to 4294967295.
DATE	A date. Format: YYYY-MM-DD. The supported range is from '1000-01-01' to '9999-12-31'
DATETIME	A date and time combination. Format: YYYY-MM-DD hh:mm:ss. The supported range is from '1000-01-01 00:00:00' to '9999-12-31 23:59:59'. Adding DEFAULT and ON UPDATE in the column definition to get automatic initialization and updating to the current date and time

https://www.w3schools.com/mysql/mysql_datatypes.asp

SQL – CREATE TABLE

Exercises

Create table and keys for a table called, in your schema

Suppliers with fields: supplier_id(INT), name(VARCHAR), phone(VARCHAR)

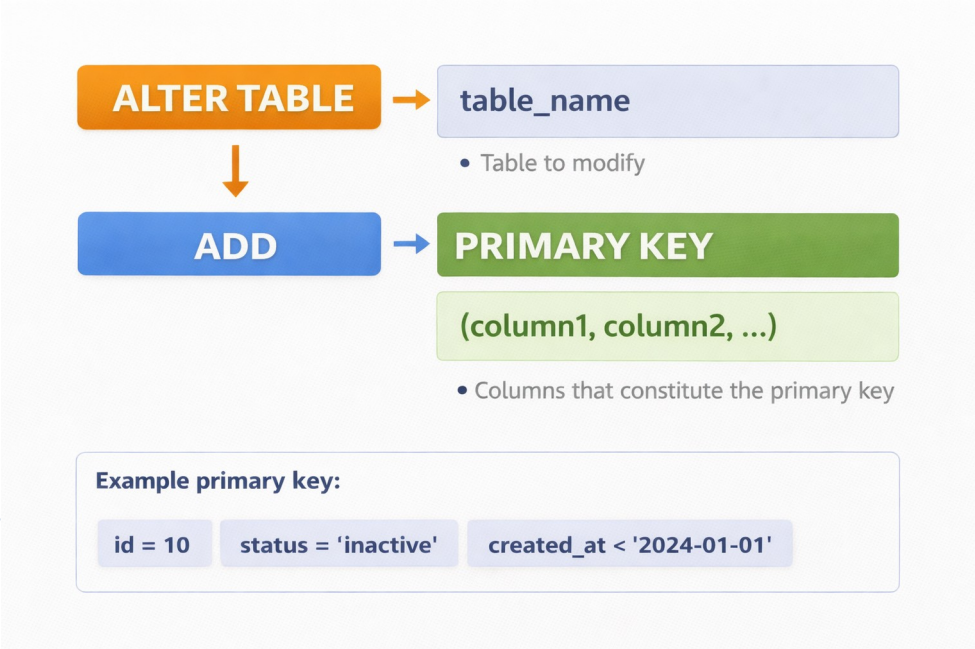
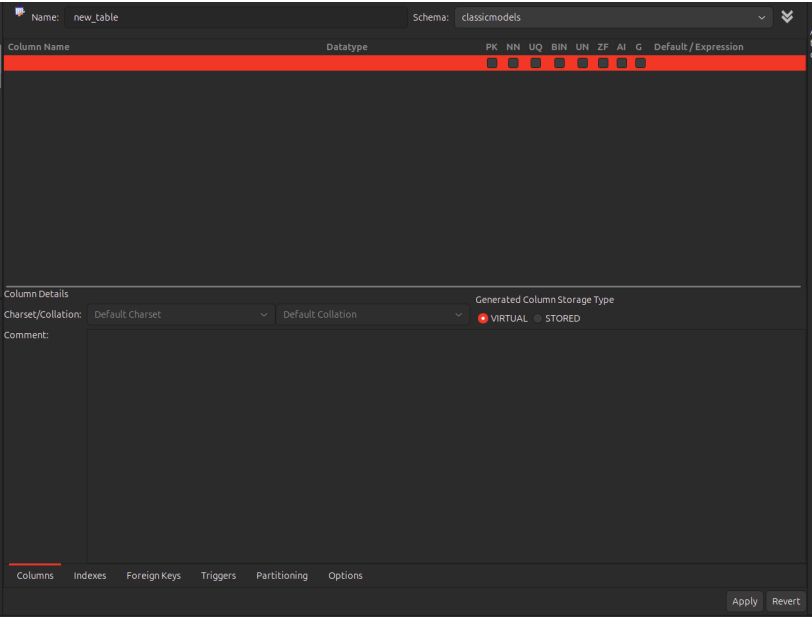
Products with fields: product_id (INT), name (VARCHAR), price (DECIMAL), supplier_id (INT)



CREATE KEYS

Create a new Primary Key

ALTER TABLE Persons
ADD PRIMARY KEY (ID);



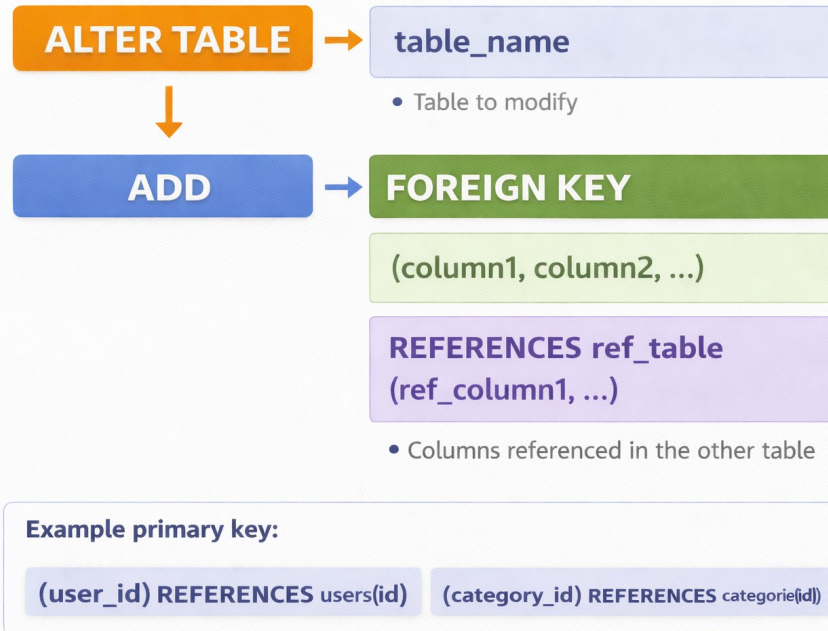
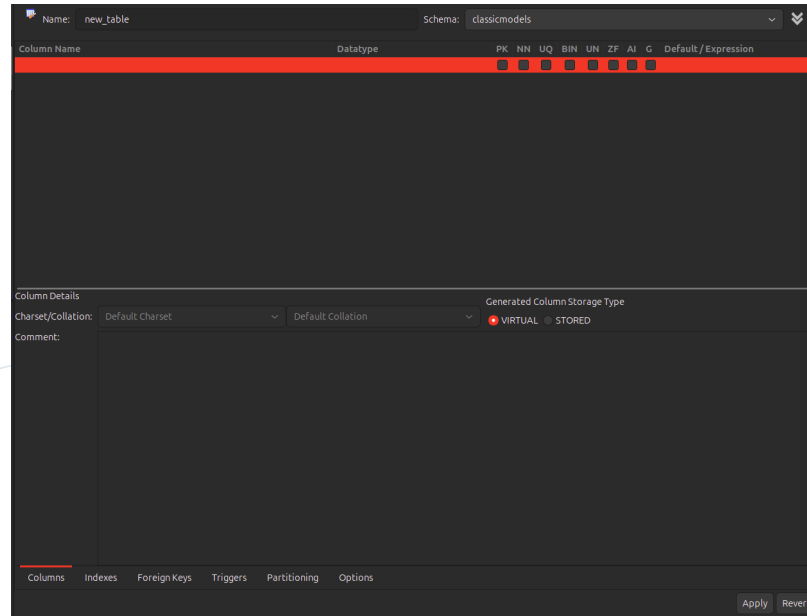
CREATE KEYS

Create a new Foreign Keys

ALTER TABLE Orders

ADD FOREIGN KEY (PersonID) REFERENCES

Persons(PersonID);



SQL – KEYS

Exercise

Primary Key

Suppliers - supplier_id

Products - product_id

Foreign Key

Products - supplier_id



SQL - INSERT

Add new entry(s) to a table

Example (go to MySQL Workbench)

```
INSERT INTO social_network.user_table  
  (surname, name, age, subscription_date)  
VALUES  
  ("Abreu", "Pedro", 23, 123456789),  
  ("Fonseca", "Rodrigo", 43, 123451544);
```

INSERT INTO

table_name (column1, column2, ...)

VALUES

(value1, value2, ...),

(value3, value4, ...),

(value5, value6, ...),

● Table and columns

● Values to insert

SQL - INSERT

Exercises

1) Add one supplier and 2 products into the table created in previous exercise.

The name of the product are “Mouse 3 buttons” and “Laptop Intel”

2) Check if the previous entries are in the table

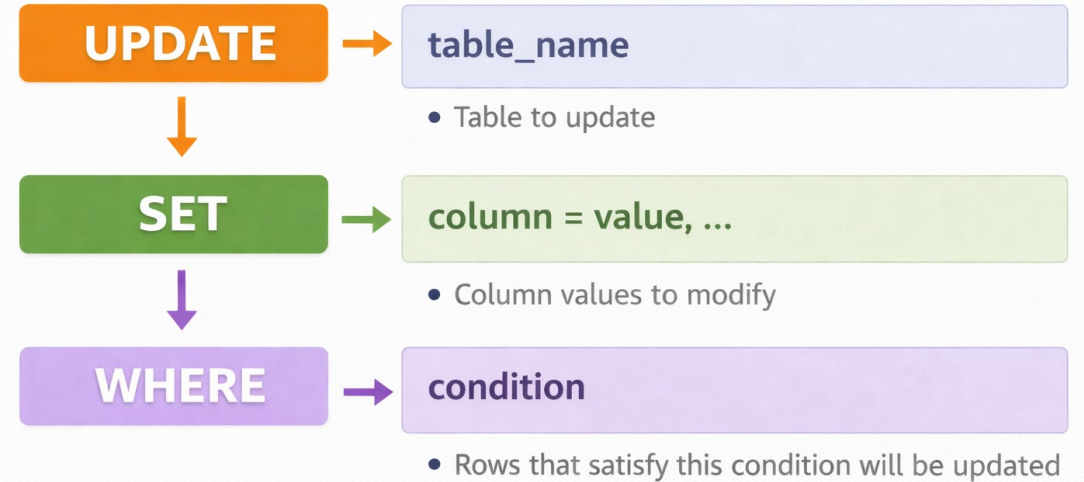


SQL - UPDATE

Update values in existing entries

Example (go to MySQL Workbench)

```
UPDATE social_network.user_table  
SET age = 30  
WHERE surname = "Abreu";
```



Example conditions:

id = 10

status = 'inactive'

created_at < '2024-01-01'

age > 18

name LIKE 'A%'

SQL - UPDATE

Exercises

- 1) Update the price to another value of the product are “Mouse 3 buttons”
- 2) Check if the previous entries are updated



SQL - DELETE

Delete entries in table

Example (go to MySQL Workbench)

```
DELETE  
FROM social_network.user_table  
WHERE surname = 'Abreu'  
LIMIT 1;
```

DELETE FROM

table_name

- Table to delete from

WHERE

condition

- Rows that satisfy this condition will be deleted

[AND | OR ...]

Additional conditions (optional)

- Combine multiple conditions

SQL - DELETE

Exercises

- 1) Delete the supplier and products created in previous exercise
- 2) Check if the previous entries are in the table



Window Functions

A window function performs an aggregate-like operation on a set of query rows. However, whereas an aggregate operation groups query rows into a single result row, a window function produces a result for each query row

Example

SELECT

customerNumber,

amount,

RANK() **OVER** (**PARTITION BY** customerNumber **ORDER BY** amount **DESC**) **AS** amount_rank

FROM classicmodels.Payments;

SELECT →

column1, window_func(column2)
OVER (**PARTITION BY** column1 **ORDER BY** column3)

FROM →

table_name

Table to query

<https://dev.mysql.com/doc/refman/8.4/en/window-functions-usage.html>

Window Functions

Aggregation Functions (window function or group by)

Name	Description
<u>AVG()</u>	Return the average value of the argument
<u>BIT_AND()</u>	Return bitwise AND
<u>BIT_OR()</u>	Return bitwise OR
<u>BIT_XOR()</u>	Return bitwise XOR
<u>COUNT()</u>	Return a count of the number of rows returned
<u>COUNT(DISTINCT)</u>	Return the count of a number of different values
<u>GROUP_CONCAT()</u>	Return a concatenated string
<u>JSON_ARRAYAGG()</u>	Return result set as a single JSON array
<u>JSON_OBJECTAGG()</u>	Return result set as a single JSON object
<u>MAX()</u>	Return the maximum value
<u>MIN()</u>	Return the minimum value
<u>STD()</u>	Return the population standard deviation
<u>STDDEV()</u>	Return the population standard deviation
<u>STDDEV_POP()</u>	Return the population standard deviation
<u>STDDEV_SAMP()</u>	Return the sample standard deviation
<u>SUM()</u>	Return the sum
<u>VAR_POP()</u>	Return the population standard variance
<u>VAR_SAMP()</u>	Return the sample variance
<u>VARIANCE()</u>	Return the population standard variance

Non-Aggregation Functions (just for windows function)

Name	Description
<u>CUME_DIST()</u>	Cumulative distribution value
<u>DENSE_RANK()</u>	Rank of current row within its partition, without gaps
<u>FIRST_VALUE()</u>	Value of argument from first row of window frame
<u>LAG()</u>	Value of argument from row lagging current row within partition
<u>LAST_VALUE()</u>	Value of argument from last row of window frame
<u>LEAD()</u>	Value of argument from row leading current row within partition
<u>NTH_VALUE()</u>	Value of argument from N-th row of window frame
<u>NTILE()</u>	Bucket number of current row within its partition.
<u>PERCENT_RANK()</u>	Percentage rank value
<u>RANK()</u>	Rank of current row within its partition, with gaps
<u>ROW_NUMBER()</u>	Number of current row within its partition

<https://dev.mysql.com/doc/refman/8.4/en/window-function-descriptions.html>

Text-to-SQL

Text to SQL is a natural language processing task that converts human-readable text queries into structured SQL queries. This lets users interact with databases using natural language.

Aspect	Text-to-SQL ✓	Sending Data in Prompt ✗
Large datasets	✓ Scales well	✗ Limited by tokens
Cost	✓ Lower	✗ Higher
Accuracy	✓ Structured	✗ Prone to errors
Real-time data	✓ Yes	✗ No
Security	✓ Safer	✗ Risky

Text-to-SQL

A good prompt should include:

- Clear instructions for what we want the model to do
- The user's query
- The database's schema: tables, columns and relationships
- Include descriptions of each element of schema

<https://platform.claude.com/cookbook/capabilities-text-to-sql-guide>



Text-to-SQL

You are an AI assistant that converts natural language queries into SQL.

Database context The name of schema is called classicmodels. The classicmodels database is a retailer of scale models of classic cars. It contains typical business data, including information about customers, products, sales orders, sales order line items, and more.

MySQL database schema

Tables:

- customers: stores customer's data.
- products: stores a list of scale model cars.
- productlines: stores a list of product lines.
- orders: stores sales orders placed by customers.
- orderdetails: stores sales order line items for every sales order.
- payments: stores payments made by customers based on their accounts.
- employees: stores employee information and the organization structure such as who reports to whom.
- offices: stores sales office data.

See the complete prompt in the classicmodels_prompt.txt file

<https://platform.claude.com/cookbook/capabilities-text-to-sql-guide>



Where to...

More SQL- CREATE USER, CREATE VIEW, VARIABLES, LOOPS, etc...

SQL Injection - Malicious attacks to database using SQL

<https://www.youtube.com/watch?v=wcaiKgQU6VE>

Prisma -

<https://www.youtube.com/watch?v=rLRIB6AF2Dg>



Make change happen

ACCREDITATIONS



MEMBERSHIPS



RANKINGS

